



Cardápio Digital Confeitaria da Re

Documentação técnica — arquitetura, estrutura, banco de dados,
segurança e operação

Versão

Fase 1 + melhorias de segurança

Data

27 de setembro de 2026

Stack

Next.js 16 · Supabase · Vercel

Sumário

- | | | | |
|----------|------------------------|-----------|------------------------------------|
| 1 | Visão geral do projeto | 9 | Regras de negócio |
| 2 | Tecnologias utilizadas | 10 | Painel administrativo |
| 3 | Arquitetura | 11 | Qualidade e testes |
| 4 | Mapa do site (rotas) | 12 | Como o projeto foi construído |
| 5 | Estrutura de pastas | 13 | Operação: rodar, publicar e manter |
| 6 | Banco de dados | 14 | Próximas fases |
| 7 | Segurança | 15 | Glossário |
| 8 | Fluxos principais | | |

Em números

144

arquivos de código

~9.000

linhas (aplicação)

12

tabelas no banco

57

testes automatizados

20

páginas / rotas

8

migrations SQL

100%

tabelas com RLS

2FA

obrigatório no painel

1. Visão geral do projeto

Cardápio digital próprio para uma confeitaria de docinhos: uma alternativa ao iFood, **sem comissão por pedido**, acessada principalmente pelo celular a partir do link na bio do Instagram.

Duas formas de venda

Delivery (pronta entrega)

Catálogo com produtos prontos, carrinho, checkout com **entrega** (bairro + taxa) ou **retirada**, pagamento na entrega/retirada (Pix ou cartão) e página de acompanhamento do pedido.

Encomendas para festa

Portfólio (galeria de trabalhos e sabores) e um formulário que monta a mensagem e abre o WhatsApp da loja. A negociação é feita pelo WhatsApp; **nada é salvo** no banco.

Público e dispositivos

- **Clientes:** visitantes sem login. Interface *mobile-first*, com fotos grandes e botões com área de toque generosa.
- **Loja:** painel administrativo em `/admin`, com login por e-mail + senha + código do app autenticador (2FA).

Fases do produto

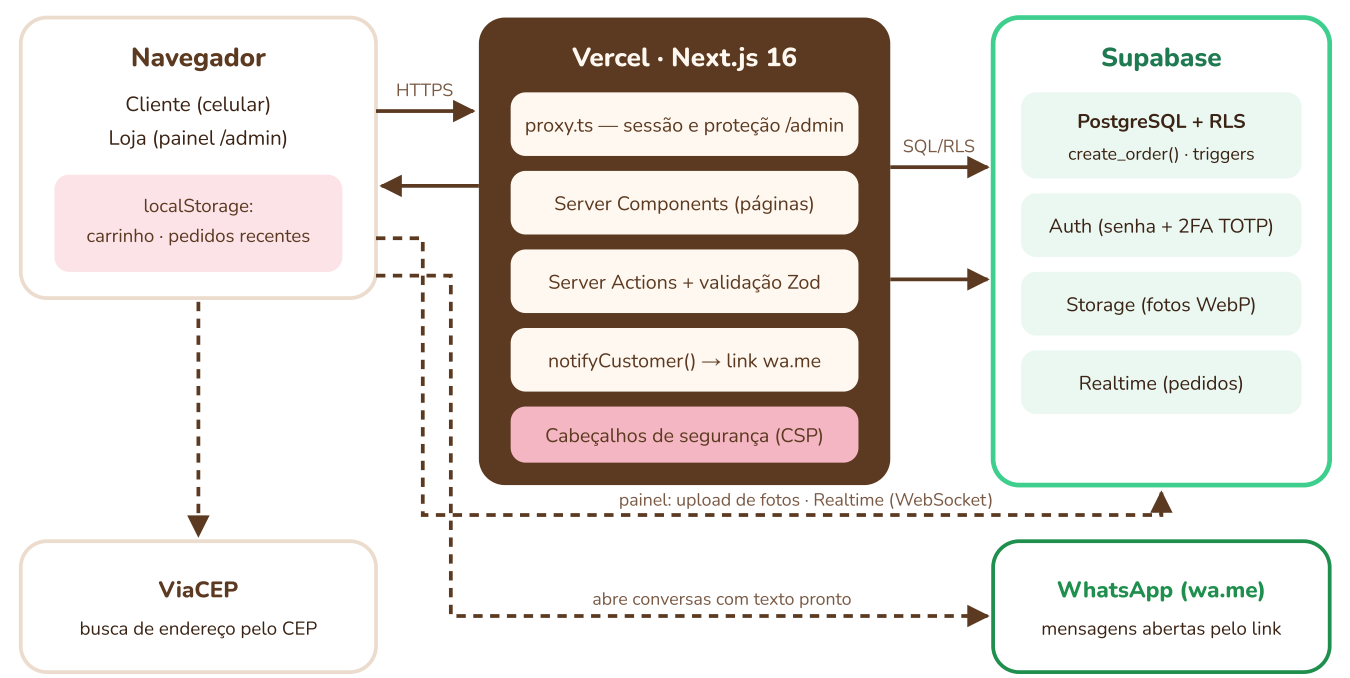
Fase	Escopo	Situação
Fase 1	Site público, pedidos, painel, tempo real, notificação por link <code>wa.me</code> , segurança, LGPD e 2FA	✅ Em produção
Fase 2	Pagamento online (Pix/cartão via gateway com webhook)	Preparada (campos no banco)
Fase 3	Mensagens automáticas pela WhatsApp Cloud API	Preparada (<code>notifyCustomer()</code>)

2. Tecnologias utilizadas

Tecnologia	Versão	Papel no projeto e por que foi escolhida
TypeScript (modo strict)	5	Linguagem de todo o código. Tipagem estrita evita erros comuns antes de rodar.
Next.js (App Router)	16.3	Framework web: páginas renderizadas no servidor (rápidas e boas para SEO), <i>Server Actions</i> para gravar dados com segurança e <code>proxy.ts</code> para proteger o painel. Turbopack como bundler.
React	19.2	Biblioteca de interface. Server Components (sem JavaScript no navegador quando possível) e Client Components para interação.
Tailwind CSS	4	Estilização por classes utilitárias, com as cores da marca como variáveis (rosa, marrom/cacau, creme).
shadcn/ui + Base UI	4 / 1.8	Componentes acessíveis (gaveta, diálogo, switch, botões) copiados para o projeto e personalizados.
Supabase	CLI 2.118	Backend completo: PostgreSQL (dados), Auth (login + 2FA), Storage (fotos) e Realtime (pedidos ao vivo). Regras de acesso por RLS no próprio banco.
@supabase/ssr + supabase-js	0.12 / 2.117	Clientes do Supabase para servidor (com cookies de sessão) e navegador.
Zod	4.6	Validação de todas as entradas (no navegador e de novo no servidor).
Zustand	5	Estado do carrinho e dos pedidos recentes, salvo no aparelho (localStorage).
Vitest	5	Testes automatizados das regras de negócio.
ESLint	9	Análise estática do código (regras do Next e do React).
lucide-react · sonner	1.48 · 2	Ícones e avisos (toasts) na tela.
Vercel	—	Hospedagem e deploy automático a cada <code>git push</code> .
ViaCEP	API pública	Preenche a rua a partir do CEP no checkout.
Fontes	—	Fraunces (títulos) e Nunito (texto), servidas pelo próprio site via <code>next/font</code> (sem chamadas ao Google).

3. Arquitetura

O navegador conversa apenas com o site na Vercel (Next.js) e, em pontos específicos, diretamente com o Supabase (upload de fotos e tempo real do painel) e com o ViaCEP. Tudo que envolve dinheiro ou dados sensíveis passa pelo servidor.



Três clientes do Supabase

Arquivo	Onde roda	Uso
<code>lib/supabase/public.ts</code>	Servidor	Leitura pública (cardápio, loja) como visitante anônimo. Sem cookies.
<code>lib/supabase/server.ts</code>	Servidor	Painel: usa a sessão do admin (cookies). Tudo passa pelo RLS.
<code>lib/supabase/client.ts</code>	Navegador	Painel: upload de fotos e Realtime, com a sessão do admin.
<code>lib/supabase/admin.ts</code>	Somente servidor	<code>service_role</code> (ignora RLS): criar pedido e ler um pedido pelo UUID. Protegido com <code>server-only</code> .

Decisões de arquitetura

- **Regras críticas no banco:** preços, totais, horário de funcionamento, transições de status e permissões são garantidos por funções, triggers e RLS no Postgres. O front apenas exibe.
- **Ponto único de notificação:** `notifyCustomer()` hoje gera o link `wa.me`; na Fase 3 será trocado pela API do WhatsApp sem alterar o restante.
- **Sem contas para clientes:** o link do pedido (UUID) é a chave de acompanhamento.
- **Dinheiro nunca em ponto flutuante:** todos os valores são inteiros em centavos.
- **Fuso fixo:** datas e horários sempre em `America/Sao_Paulo`, independentemente do servidor.

4. Mapa do site (rotas)

Site público sem login

Caminho	O que faz
/	Home: logo, nome, selo “Aberto agora/Fechado”, atalho “Acompanhar meu pedido”, botões Delivery e Encomendas para festa , horários, endereço de retirada, WhatsApp, Instagram e link da política de privacidade.
/pronta-entrega	Catálogo (Delivery): abas fixas por categoria, cards com foto e preço, selo “Esgotado”, gaveta do produto (quantidade + observação), carrinho flutuante.
/checkout	Dados da cliente, entrega (CEP via ViaCEP, bairro com taxa) ou retirada, pagamento (Pix/cartão na entrega), observações, resumo, pedido mínimo, aviso LGPD.
/pedido/[id]	Confirmação e acompanhamento: número, status, linha do tempo, itens e total; botão “Acompanhar pelo WhatsApp”; atualiza a cada 30 s. [id] = UUID.
/encomendas	Apresentação, quantidade mínima e antecedência, galeria com lightbox, sabores e formulário que gera a mensagem do WhatsApp.
/privacidade	Política de Privacidade (LGPD) com dados reais da loja.

Painel administrativo login + 2FA

Caminho	O que faz
/admin/login	E-mail e senha (com botão para mostrar a senha).
/admin/2fa/configurar	Primeiro acesso: ativa o 2FA (botão para o app Senhas do iPhone, QR code ou chave).
/admin/2fa/verificar	Login do dia a dia: código de 6 números do app autenticador.
/admin/pedidos	Lista com filtros por status, atualizada em tempo real, alerta sonoro.
/admin/pedidos/[id]	Detalhe, avançar/cancelar status, “Avisar cliente”, histórico.
/admin/produtos · /novo · /[id]	Categorias e produtos: criar, editar, foto, ordenar, ativar, esgotado com um toque.
/admin/encomendas · /sabores/...	Sabores (com diferenciais) e galeria de fotos (upload múltiplo, legendas).
/admin/bairros	Bairros atendidos e taxas de entrega.
/admin/configuracoes	Interruptor “Receber pedidos”, dados da loja, logo, horários, encomendas, mensagens de WhatsApp.

Arquivos especiais

layout.tsx (estrutura e fontes), error.tsx e not-found.tsx (páginas de erro), loading.tsx (esqueleto do catálogo), icon.png, apple-icon.png e favicon.ico (ícones gerados a partir do batedor da logo), proxy.ts (antigo *middleware* no Next 16).

5. Estrutura de pastas

```
Confeitaria/
├── src/
│   ├── app/                                # rotas (App Router)
│   │   ├── page.tsx                        # Home
│   │   ├── pronta-entrega/ checkout/ pedido/[id]/ encomendas/ privacidade/
│   │   ├── admin/
│   │   │   ├── login/ 2fa/configurar/ 2fa/verificar/
│   │   │   └── (protected)/                # tudo aqui exige admin com 2FA
│   │   │       ├── pedidos/ produtos/ encomendas/ bairros/ configuracoes/
│   │   │       └── layout.tsx              # menu, "Ao vivo", som, sair
│   │   ├── layout.tsx globals.css error.tsx not-found.tsx
│   │   └── icon.png apple-icon.png favicon.ico
│   ├── components/
│   │   ├── public/                        # catálogo, carrinho, checkout, galeria...
│   │   ├── admin/                         # upload de foto, toggle, realtime, reordenar...
│   │   └── ui/                             # shadcn/ui (botão, gaveta, diálogo...)
│   ├── lib/
│   │   ├── data/                          # leituras no servidor (loja, catálogo, pedidos)
│   │   ├── domain/                        # regras puras: status, horários, carrinho, preços
│   │   ├── validation/                    # schemas Zod (checkout e painel)
│   │   ├── notifications/                 # notifyCustomer() e textos das mensagens
│   │   ├── supabase/                     # clientes e tipos do banco
│   │   ├── whatsapp/                     # links wa.me e mensagem de encomenda
│   │   ├── admin/                         # compressão de imagem, som, Storage
│   │   ├── security/                      # hash do IP (limite de pedidos)
│   │   └── auth.ts format.ts site-url.ts viacep.ts ...
│   ├── stores/                            # Zustand: carrinho e pedidos recentes
│   └── proxy.ts                           # protege /admin e renova a sessão
├── supabase/
│   ├── migrations/                        # 8 arquivos SQL (schema → 2FA)
│   └── seed.sql                           # dados de exemplo
├── tests/                                 # 9 arquivos, 57 testes (Vitest)
├── public/                               # Logo.png
├── next.config.ts                         # imagens + cabeçalhos de segurança (CSP)
├── .env.example README.md package.json
```

Organização por responsabilidade

lib/domain

Regras de negócio **puras** (sem banco, sem tela):
fluxo de status, horários, carrinho e preços.
Fáceis de testar e reutilizar.

lib/data

Funções que **leem** do banco no servidor. As
páginas só chamam essas funções.

app/**/actions.ts

Server Actions que gravam dados: validam com
Zod, conferem o admin e chamam o banco.

components

Interface separada em **public** (clientes), **admin**
(painel) e **ui** (componentes base).

6. Banco de dados

PostgreSQL no Supabase. Valores monetários em **centavos** (inteiros). Todas as tabelas com **RLS** habilitado.

Tabelas

Tabela	Conteúdo	Acesso público
<code>store_settings</code>	Linha única: nome, slogan, logo, WhatsApp, Instagram, endereço de retirada, interruptor “receber pedidos”, pedido mínimo, textos e mínimos de encomenda.	Leitura
<code>opening_hours</code>	7 linhas (dom–sáb): fechado?, abre, fecha.	Leitura
<code>whatsapp_templates</code>	Texto da mensagem para cada status, com placeholders.	—
<code>categories</code>	Categorias do catálogo (nome, slug, ordem, ativa).	Leitura (ativas)
<code>products</code>	Produtos: categoria, nome, descrição, <code>price_cents</code> , foto, ordem, ativo, disponível (esgotado).	Leitura (ativos)
<code>delivery_zones</code>	Bairros atendidos e taxa (<code>fee_cents</code>).	Leitura (ativos)
<code>orders</code>	Pedido: número sequencial (a partir de 100), cliente, recebimento, endereço, subtotal/taxa/total, pagamento, <code>payment_status</code> (Fase 2), status, quer acompanhar pelo WhatsApp, hash do IP.	Nenhum
<code>order_items</code>	Itens com snapshot de nome e preço no momento do pedido.	Nenhum
<code>order_status_history</code>	Cada mudança de status com data/hora e quem mudou.	Nenhum
<code>custom_flavors</code>	Sabores das encomendas: descrição, foto, diferenciais.	Leitura (ativos)
<code>custom_gallery</code>	Fotos da galeria de trabalhos e legendas.	Leitura (ativas)
<code>admin_users</code>	Quais usuários do Auth são administradores.	—

Tipos (enums): `fulfillment_type` (delivery, pickup) · `order_status` (received, confirmed, out_for_delivery, ready_for_pickup, completed, cancelled) · `payment_method` · `payment_status` (pending, paid, refunded, not_applicable).

Funções e triggers

Nome	O que faz
<code>create_order(payload)</code>	Cria o pedido numa transação única : confere loja aberta, limite por IP, produtos ativos/disponíveis, bairro válido e pedido mínimo; recalcula subtotal, taxa e total a partir do banco; grava pedido e itens. Só a <code>service_role</code> pode executar.
<code>is_store_open_now()</code>	“Aberto agora” = interruptor ligado e dentro do horário do dia (fuso de São Paulo).
<code>is_admin_member()</code>	O usuário está em <code>admin_users</code> ? (usado no fluxo de login)

Nome	O que faz
<code>is_admin()</code>	Membro e sessão com 2FA (<code>aa12</code>). Usada em todas as regras RLS e do Storage.
<code>validate_order_status_transition</code>	Trigger: bloqueia mudanças de status fora do fluxo permitido.
<code>log_order_status</code>	Trigger: registra o histórico de status automaticamente.
<code>set_updated_at</code>	Trigger: atualiza <code>updated_at</code> em cada alteração.

Storage e Realtime

- **Bucket `images`**: leitura pública pela URL, **sem listagem pública**; upload/alteração/exclusão apenas para admin; até 5 MB; WebP, JPEG ou PNG. Pastas: `products/`, `flavors/`, `gallery/`, `store/`.
- **Realtime**: a tabela `orders` está na publicação `supabase_realtime`; o RLS garante que só admins recebem os eventos.

Migrations

Arquivo	Conteúdo
<code>20260926000001_schema.sql</code>	Tipos e tabelas
<code>20260926000002_functions.sql</code>	Funções (<code>create_order</code> , horários) e triggers
<code>20260926000003_rls.sql</code>	Políticas RLS e permissões por coluna
<code>20260926000004_storage_realtime.sql</code>	Bucket de fotos e Realtime
<code>20260926000005_grants.sql</code>	Permissões explícitas (projeto sem “expose” automático)
<code>20260927000006_storage_admin_select.sql</code>	Leitura do bucket para admin (necessária para excluir fotos)
<code>20260927000007_templates_tracking_link.sql</code>	Placeholder <code>{link}</code> nas mensagens
<code>20260927000008_admin_requires_mfa.sql</code>	<code>is_admin()</code> passa a exigir 2FA (<code>aa12</code>)

7. Segurança

Princípio: **nunca confiar no navegador**. As proteções estão em camadas — interface, servidor e banco — e a camada final (RLS no Postgres) vale mesmo que alguém acesse o banco diretamente com a chave pública.

Matriz de permissões (RLS)

Dado	Visitante	Logado sem 2FA	Admin com 2FA	Servidor
Catálogo, sabores, galeria, bairros	Lê ativos	Lê ativos	Tudo	Tudo
Configurações e horários	Lê	Lê	Lê e altera	Tudo
Mensagens de WhatsApp	—	—	Tudo	Tudo
Pedidos e itens	Nada	Nada	Lê; altera só o status	Cria (<code>create_order</code>)
Fotos (Storage)	Vê pela URL	Vê pela URL	Envia/exclui	—

Proteções implementadas

Preços à prova de fraude

O navegador envia só produto e quantidade. Preço, taxa e total são recalculados no banco. Itens guardam snapshot de nome e preço.

Pedidos inalteráveis

Público não lê nem altera pedidos. O admin só muda o status (permissão por coluna); trigger valida a ordem das etapas.

Limite de pedidos

Máximo de 3 pedidos por IP a cada 10 minutos. O IP é guardado só como hash SHA-256 com sal. Na Vercel o IP não pode ser falsificado.

Login em duas etapas

Senha + código TOTP (app Senhas do iPhone). Sem o código a sessão é `aa11` e o banco não libera nada do painel.

Chave secreta isolada

A `service_role` só existe no servidor (`server-only`) e como variável **Secret** na Vercel.

Validação em tudo

Zod no navegador (mensagens por campo) e de novo no servidor. Fotos restritas à pasta certa (sem `../`).

Cabeçalhos HTTP

CSP (só o próprio site, Supabase e ViaCEP), anti-clickjacking, HSTS, `nosniff`, `Referrer-Policy`, `Permissions-Policy`.

Link do pedido

UUID (122 bits aleatórios), fora do Google (`noindex`), mostra só dados necessários.

LGPD e cookies

- Página `/privacidade`: dados coletados, finalidade, base legal, compartilhamento, retenção, direitos e contato.
- O site público **não usa cookies**; o carrinho fica no aparelho. Cookies apenas na sessão do painel (estritamente necessários) — por isso não há banner de consentimento.
- Se forem adicionados Meta Pixel, Google Analytics ou similares, será necessário banner de consentimento e atualização da política.

Verificações realizadas: npm audit sem vulnerabilidades em dependências de produção; nenhum HTML injetado sem escape; chave secreta usada apenas no servidor; testes de RLS por papel (visitante, logado sem 2FA, admin com 2FA).

8. Fluxos principais

8.1 Pedido de delivery

- 1 A cliente escolhe produtos em `/pronta-entrega`. O carrinho guarda só **produto, quantidade e observação** no aparelho.
- 2 No `/checkout` preenche dados, entrega/retirada e pagamento. O CEP consulta o ViaCEP e seleciona o bairro automaticamente.
- 3 A Server Action `createOrder` valida com Zod e calcula o hash do IP.
- 4 O servidor chama `create_order()` no banco, que confere tudo e **recalcula os valores** numa transação.
- 5 A cliente vai para `/pedido/[uuid]`; o pedido fica salvo no aparelho para o atalho “Acompanhar meu pedido”.
- 6 O painel recebe o pedido **em tempo real** com som e aviso.

8.2 Status e notificação

- 1 A loja avança o status no painel (ex.: Recebido → Confirmado). O trigger valida e registra o histórico.
- 2 O botão “Avisar cliente” chama `notifyCustomer()`, que monta o texto do status (placeholders `{nome}`, `{numero}`, `{total}`, `{loja}`, `{endereco_retirada}`, `{link}`).
- 3 Abre o WhatsApp da cliente com a mensagem pronta, incluindo o link de acompanhamento.
- 4 A página do pedido da cliente se atualiza sozinha a cada 30 segundos.

```
// Fluxo de status (espelhado no banco e em lib/domain/order-status.ts)
Entrega: Recebido → Confirmado → Saiu para entrega → Concluído
Retirada: Recebido → Confirmado → Pronto para retirada → Concluído
Cancelado: possível em qualquer etapa antes de Concluído
```

8.3 Login do painel (2FA)

- 1 E-mail e senha → sessão `aa11`. O sistema confere se o usuário é admin.
- 2 Primeiro acesso: tela de configuração (app Senhas do iPhone via link `otpauth://`, QR code ou chave).
- 3 Nos próximos acessos: tela do código de 6 números (o iPhone sugere o código no teclado).
- 4 Código correto → sessão `aa12` → o RLS libera o painel.

8.4 Upload de fotos

A foto é **redimensionada (1600 px) e convertida para WebP no próprio celular** ($\approx 700 \text{ KB} \rightarrow 36 \text{ KB}$ no teste), enviada direto ao Storage com a sessão do admin, e o caminho é gravado via Server Action. Fotos substituídas ou excluídas são apagadas do Storage.

8.5 Encomendas para festa

O formulário (nome, data, quantidade, sabores, tema) só monta a mensagem e abre `wa.me`. Avisa — sem bloquear — se a data não respeita a antecedência mínima ou se a quantidade está abaixo do mínimo.

9. Regras de negócio

Regra	Como funciona	Onde é garantida
Loja aberta	Interruptor “Receber pedidos” ligado e dentro do horário do dia. Fechada: catálogo visível, finalização bloqueada.	<code>is_store_open_now()</code>
Pedido mínimo	Subtotal (sem taxa) $\geq$ valor configurado.	<code>create_order()</code>
Taxa de entrega	Por bairro cadastrado; retirada é grátis. “Outros bairros” cadastrado como opção com taxa própria.	<code>create_order()</code>
Produto esgotado	Aparece com selo e não pode ser pedido.	<code>create_order()</code>
Pagamento	Pix ou cartão, na entrega/retirada. Dinheiro desativado (reativável em <code>lib/domain/payment.ts</code>).	Zod no servidor
Quantidade	1 a 99 por item; até 50 itens por pedido.	Zod + banco
Status	Somente as transições do fluxo; concluído/cancelado são finais.	Trigger
Limite de pedidos	3 por IP a cada 10 minutos.	<code>create_order()</code>
Encomendas	Quantidade mínima e antecedência em dias (avisos, sem bloqueio).	Interface
Acompanhamento	Atalho na Home por 3 dias enquanto o pedido estiver em andamento; link nas mensagens de status.	localStorage + templates

10. Painel administrativo

Pedidos

- Lista em **tempo real** (Supabase Realtime) com indicador “Ao vivo/Offline”.
- Alerta: som (“Ativar som”), vibração, aviso na tela, contador no título da aba e notificação do sistema.
- Filtros: Em andamento, por status, Todos (100 mais recentes).
- Detalhe com cliente, endereço, itens, observações, pagamento e histórico.
- Avançar status, cancelar (com confirmação) e “Avisar cliente”.

Produtos e categorias

- Criar, editar, ocultar, ordenar (setas ↑↓) e excluir.
- Foto com compressão automática.
- **Disponível/Esgotado** com um toque.
- Pedidos antigos preservam nome e preço (snapshot).

Encomendas

- Sabores com descrição, foto e diferenciais.
- Galeria: várias fotos de uma vez, legenda, ordenar, ocultar e excluir.

Bairros

- Adicionar, editar nome e taxa, ativar/desativar, ordenar e excluir.

Configurações

- Interruptor **Receber pedidos** (fechar a loja na hora).
- Dados da loja: nome, slogan, WhatsApp, Instagram, endereço, pedido mínimo e logo.
- Horários por dia da semana; textos, mínimo e antecedência das encomendas.
- Mensagens de WhatsApp para cada status, com placeholders.

O alerta de pedido novo depende do painel aberto e da tela ligada (tablet ou computador durante o expediente). Celular bloqueado não recebe o alerta.

11. Qualidade e testes

Testes automatizados (Vitest) — 57 testes em 9 arquivos

Arquivo	O que verifica
<code>format.test.ts</code>	Moeda BRL, máscaras de telefone e CEP, datas no fuso de São Paulo
<code>store-hours.test.ts</code>	Aberto/fechado, “Abre terça às 10h”, agrupamento de horários, fuso
<code>order-status.test.ts</code>	Fluxos de entrega/retirada, cancelamento, etapas não puláveis
<code>cart.test.ts</code>	Preço vindo do catálogo, itens indisponíveis, limites de quantidade
<code>order-validation.test.ts</code>	Checkout: campos obrigatórios, telefone, CEP, dinheiro recusado, payload sem preços
<code>admin-validation.test.ts</code>	Painel: preço, WhatsApp, Instagram, horários, diferenciais, caminho de foto
<code>custom-order-message.test.ts</code>	Mensagem de encomenda, datas e antecedência
<code>notifications.test.ts</code>	Mensagens de status, placeholders e link
<code>recent-orders.test.ts</code>	Atalho “Acompanhar meu pedido”, limites e prazo

Testes do banco

Durante o desenvolvimento, as migrations foram executadas num Postgres embutido (PGLite) com simulação do ambiente Supabase, validando: cálculo de totais, snapshot de itens, pedido mínimo, bairro, esgotado, limite por IP, loja fechada, fluxo de status, histórico, RLS por papel (visitante, logado sem 2FA, admin com 2FA) e permissões com e sem “expose” automático. *(Script de teste não versionado no repositório.)*

Outras verificações

- `npm run typecheck` — TypeScript estrito sem erros.
- `npm run lint` — ESLint sem avisos.
- `next build` — build de produção validado antes de cada entrega.
- Testes manuais no navegador em tamanho de celular (375 px) a cada etapa.

12. Como o projeto foi construído

O trabalho seguiu etapas pequenas e testáveis, com aprovação ao final de cada uma.

Etapa	Entregas
Planejamento	Estrutura de pastas, schema SQL com RLS e ordem de implementação aprovados.
(a) Base	Next.js 16 + TypeScript + Tailwind + shadcn; projeto Supabase; migrations; seed; clientes Supabase; <code>proxy.ts</code> ; testes iniciais.
(b) Catálogo	Home, catálogo com abas, gaveta do produto, carrinho persistente.
(c) Checkout	Formulário com Zod, ViaCEP, <code>create_order()</code> , limite por IP, página de confirmação/acompanhamento.
(d) Encomendas	Galeria com lightbox, sabores, formulário que gera mensagem de WhatsApp.
(e) Painel	Login, pedidos, produtos/categorias, sabores/galeria, bairros, configurações, upload com compressão.
(f) Tempo real	Realtime com alerta sonoro, <code>notifyCustomer()</code> e botão “Avisar cliente”, README.
Ajustes	Nome “Confeitaria da Re”, botão “Delivery”, logo, favicon, “Outros bairros”, remoção do pagamento em dinheiro.
Segurança e LGPD	Cabeçalhos de segurança/CSP, política de privacidade, acompanhamento sem conta, login em duas etapas (2FA), botão para mostrar a senha.
Publicação	Deploy na Vercel com variáveis Config/Secret; 2FA configurado.

Histórico de commits

```
aeb2846 feat: cardápio digital da confeitaria – Fase 1
31c678d feat: favicon e ícone do app com o batedor da logo
b698d61 feat: remove dinheiro, adiciona cabeçalhos de segurança e política de privacidade
801ca88 feat: acompanhamento do pedido sem conta (atalho na Home e link nas mensagens)
afa9962 feat: login em duas etapas, olho na senha e link de acompanhamento
```

Problemas encontrados e soluções

Problema	Solução
Excluir foto não apagava o arquivo	O Storage exige permissão de leitura além de exclusão; migration 0006 libera leitura só ao admin.
React 19 limpava o formulário após erro	Formulários do painel enviados por <code>onSubmit</code> , preservando o que foi digitado.
Teclado cobria a foto ao abrir produto no celular	Foco automático só com teclado/mouse, não com toque.
Fuso horário do servidor (UTC)	Todas as datas calculadas explicitamente em <code>America/Sao_Paulo</code> .
Vercel recusou variáveis <code>NEXT_PUBLIC_</code> como Secret	Públicas como Config , chave secreta e sal como Secret .

Problema	Solução
CSS parava de atualizar no <code>npm run dev</code> (Windows)	Reiniciar o servidor de desenvolvimento.

13. Operação: rodar, publicar e manter

Rodar localmente

```
npm install
cp .env.example .env.local          # preencher as variáveis
npx supabase login
npx supabase link --project-ref SEU_PROJECT_REF
npm run db:seed                     # só na primeira vez
npm run dev                         # http://localhost:3000
```

Variáveis de ambiente

Variável	Vercel	Descrição
NEXT_PUBLIC_SUPABASE_URL	Config	Endereço do projeto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Config	Chave pública (publishable)
SUPABASE_SERVICE_ROLE_KEY	Secret	Chave secreta — somente servidor
RATE_LIMIT_SALT	Secret	Sal do hash de IP
SITE_URL (opcional)	Config	Domínio próprio para o <code>{link}</code> ; na Vercel é detectado sozinho

Comandos

Comando	Uso
npm test	Testes automatizados
npm run typecheck · npm run lint	Verificação de tipos e estilo
npm run db:push	Aplica migrations novas no Supabase
npm run db:types	Regenera os tipos do banco

Publicação

- Cada `git push` na branch principal gera um deploy na Vercel.
- Ordem com migrations:** publique o código antes de aplicar uma migration que dependa dele (ex.: 2FA).

Tarefas de manutenção

Situação	O que fazer
Novo administrador	Supabase → Authentication → Users → Add user (Auto Confirm) → SQL: <code>insert into admin_users ...</code> → primeiro login configura o 2FA.
Admin perdeu o celular	Supabase → Authentication → Users → usuário → remover fator MFA.

Situação	O que fazer
Novo serviço no navegador	Adicionar o domínio na CSP em <code>next.config.ts</code> .
Mudança no uso de dados	Atualizar <code>/privacidade</code> e a data da política.

Custos de uso comercial: o plano gratuito da Vercel é para uso não comercial (recomendado Pro) e o plano gratuito do Supabase pausa projetos inativos e não tem backup diário (recomendado Pro). Verifique valores e termos atuais nos sites.

14. Próximas fases

Fase 2 — Pagamento online

- `orders` já possui `payment_status`, `payment_provider` e `payment_reference`.
- Gerar a cobrança (Pix/cartão) no gateway após o `createOrder`.
- Criar um Route Handler de **webhook** que valida a assinatura do gateway e marca o pedido como pago — nunca confiar no navegador.
- Adicionar o domínio do gateway na CSP e atualizar a política de privacidade.

Fase 3 — WhatsApp Cloud API

- Trocar apenas o final de `notifyCustomer()` pela chamada à API, retornando `{ channel: 'whatsapp_api', sent: true }`; o painel já trata esse retorno.
- Envio automático a cada status para quem optou por acompanhar (`wants_whatsapp_updates`).

Melhorias possíveis

- Notificações push para o alerta de pedidos com o celular bloqueado.
- Paginação/busca por data nos pedidos antigos (hoje: 100 mais recentes).
- Botão “Anonimizar dados da cliente” (pedidos de exclusão pela LGPD).
- Anti-robô (Cloudflare Turnstile) se surgirem pedidos falsos.

15. Glossário

Termo	Significado
App Router	Sistema de rotas do Next.js baseado em pastas dentro de <code>src/app</code> .
Server Component	Componente React renderizado no servidor; não envia JavaScript ao navegador.
Server Action	Função que roda no servidor chamada por um formulário/botão; usada para gravar dados com segurança.
Proxy (proxy.ts)	Código que roda antes das páginas; no Next 16 substitui o antigo “middleware”.
RLS	<i>Row Level Security</i> : regras no Postgres que definem quais linhas cada usuário pode ler/alterar.
service_role	Chave do Supabase com acesso total (ignora RLS); usada só no servidor.
Migration	Arquivo SQL versionado que altera a estrutura do banco de forma reproduzível.
Seed	Dados iniciais de exemplo.
Snapshot	Cópia do nome e preço do produto guardada no item do pedido.
UUID	Identificador aleatório de 128 bits, impossível de adivinhar; usado no link do pedido.
Realtime	Recurso do Supabase que envia mudanças do banco ao navegador por WebSocket.
2FA / TOTP	Verificação em duas etapas com código de 6 dígitos que muda a cada 30 s.
AAL1 / AAL2	Nível da sessão: só senha (aal1) ou senha + código (aal2).
CSP	<i>Content Security Policy</i> : cabeçalho que limita de onde o navegador carrega e envia dados.
Zod	Biblioteca de validação de dados com mensagens por campo.
localStorage	Armazenamento do navegador no próprio aparelho (carrinho e pedidos recentes).
wa.me	Link oficial do WhatsApp que abre uma conversa com texto pronto.
LGPD	Lei Geral de Proteção de Dados (Lei 13.709/2018).